NAME:TANKISO MASOEBE

OPERATING SYSTEM 10

1. Protocol Comparison Table: Mutual Exclusion Algorithms

Mutual exclusion is a technique used in distributed systems to ensure that only one process at a time can access a shared resource or critical section. This prevents conflicts, data corruption, and inconsistent results.

The three algorithms compared below are Centralized, Distributed (Ricart–Agrawala), and Token-Ring mutual exclusion.

1. Detailed Comparison Table

Feature	Centralized Algorithm	Distributed (Ricart–Agrawala)	Token-Ring Algorithm
How it works	A central coordinator grants permission to processes that want to enter the critical section.	Processes communicate directly by sending request and reply messages. Timestamps determine which process gets priority.	Processes are arranged in a logical ring, and a special message called a token circulates between them. Only the process holding the token can enter the critical section.
Messages required	Normally 3 messages per access: request, grant, and release.	Normally 2(N − 1) messages per access: \(N-1\) requests and \(N-1\) replies, where N is the number of processes.	Approximately 1 to N messages to obtain the token, depending on its position in the ring. An additional token-transfer message is needed after use.
Advantages	Simple to implement, easy to manage, and has low message overhead.	No central coordinator is required, and access is ordered using timestamps.	Simple access control, predictable token circulation, and no need for permission replies from every process.
Drawbacks	The coordinator can become a bottleneck when many	Generates many messages as the number of processes	Token circulation can introduce delays, especially when the requesting

	processes request access.	increases. Every other process must reply.	process is far from the token.
Failure scenarios	If the coordinator fails, processes may be unable to enter the critical section until a new coordinator is elected.	If a process fails or does not reply, requests may block indefinitely unless failure detection and recovery mechanisms are implemented.	If the token is lost, no process can enter the critical section until a new token is safely created. A failed node can also disrupt the ring.
Scalability	Suitable for small or moderately sized systems.	Less efficient for large systems because the number of messages increases with the number of processes.	Can work well in systems with a stable membership and predictable communication pattern.
Real-world use case	A small network server controlling access to a shared printer or database resource.	A distributed database in which several servers need coordinated access to a shared logical resource without relying on a central coordinator.	A ring-based distributed computing system or a controlled network of devices that passes access permission in a fixed order.

2. Explanation of Message Complexity

Let N represent the total number of processes in the system.

- Centralized: Requires approximately 3 messages per completed access: request, grant, and release.
- Ricart–Agrawala: Requires $2(N-1)$ messages per access because a process requests permission from every other process and receives a reply from each one.
- Token-Ring: Requires between 1 and N token transmissions to reach a requesting process, depending on the token's location and the ring's size. Token passing continues after the critical section is released.

Note: These counts assume normal operation without failures and use the conventional message-counting model. Token-Ring counts vary with how the initial token position and subsequent token release are counted.

3. Conclusion

The Centralized algorithm is suitable for systems that need a simple solution with low message overhead, but it depends on a central coordinator. Ricart–Agrawala is useful

when decentralized coordination is preferred, although it requires more messages and can be affected by unresponsive processes. Token-Ring is appropriate when processes can be organized into a stable logical ring, but token loss and node failures must be handled carefully.

2. The Raft Consensus Algorithm

The Raft consensus algorithm is a distributed consensus algorithm designed to help multiple computers agree on the same data, even when some computers fail. It is considered an alternative to Paxos, which is known for being difficult to understand and implement. Raft is commonly used in distributed systems that require consistency and reliability.

How Raft Works

Raft uses a leader-based approach to coordinate communication between servers. Each server can be in one of three states: leader, follower, or candidate. The leader manages client requests and coordinates the replication of data across the other servers. Followers receive updates from the leader, while candidates compete to become the leader when an election is necessary.

When a server starts, it is usually a follower. If it does not receive communication from a leader within a specific time, it becomes a candidate and starts a leader election by requesting votes from other servers. A candidate that receives votes from a majority of the servers becomes the leader. This process helps ensure that only one leader is recognized for a particular term.

Log Replication Mechanism

Raft maintains a replicated log containing commands or operations requested by clients. When a client sends a request, the leader records the command in its local log and sends it to the follower servers. Once a majority of servers have stored the log entry, the leader marks it as committed and applies it to its state machine. The followers then apply the committed entry, ensuring that the servers maintain consistent data.

If a server fails, Raft can recover when it reconnects by receiving missing log entries from the leader. Raft also uses terms and log consistency checks to prevent conflicting entries from being committed.

Why Raft Is Easier to Understand Than Paxos

Raft is considered more understandable because it divides consensus into clear components: leader election, log replication, and safety. Its strong leader model simplifies the explanation of how decisions are coordinated. In comparison, Paxos

involves more abstract concepts and variations that can make its operation harder to follow.

In conclusion, Raft provides a practical way to achieve consensus in distributed systems while maintaining consistency, fault tolerance, and a relatively straightforward design.

3. Here is a college-level report on etcd and the Raft consensus algorithm, covering how it provides reliable distributed coordination, locks, and leader election.

Case Study: etcd and the Raft Consensus Algorithm

Introduction

etcd is an open-source, distributed key-value store designed to store important configuration data and coordinate distributed applications. It is widely used in cloud computing environments, particularly by Kubernetes, to maintain reliable cluster information. etcd uses the Raft consensus algorithm to ensure that multiple servers agree on updates and maintain consistent data, even when some servers fail.

How etcd Uses Raft

Raft allows an etcd cluster to operate through a leader-based approach. One server acts as the leader, while the remaining servers act as followers. The leader manages operations that require agreement across the cluster and replicates updates to followers.

When a client submits a request to modify data, the leader records the operation in its log and sends the entry to the other servers. Once a majority of the cluster members have stored the entry, it can be committed and applied to the key-value store. This process ensures that successful updates are consistently recorded across the cluster.

If the leader fails, the remaining servers can elect a new leader through a voting process. This allows the cluster to continue operating as long as a majority of its members can communicate. For example, a cluster of three servers can tolerate one server failure while maintaining consensus. Source: etcd project documentation

Distributed Locks

etcd provides distributed locking mechanisms that prevent multiple applications from accessing the same shared resource simultaneously. A client requests a lock, and etcd

creates a key representing ownership. Other clients must wait until the lock becomes available.

etcd uses leases to manage lock ownership. A lease has a time-to-live period and can be kept active through periodic communication. If the lease expires, the associated lock is automatically released. This helps prevent resources from remaining locked indefinitely when a client crashes or disconnects. Source: etcd concurrency API

Leader Election

etcd also supports distributed leader election. Multiple application instances can participate in an election, but only one becomes the leader for a particular election. The selected leader can coordinate tasks such as scheduling jobs, managing services, or performing background operations.

If the leader resigns or its lease expires, another participant can acquire leadership. This reduces dependence on a single permanent application instance and improves fault tolerance.

Conclusion

etcd demonstrates how Raft can provide reliable coordination in distributed systems. Through replicated logs, majority voting, distributed locks, leases, and leader election, it helps applications maintain consistent information and recover from failures. Therefore, etcd is a valuable solution for modern cloud platforms and distributed applications that require reliable coordination.

References

1. etcd Project. *Distributed Reliable Key-Value Store*. https://github.com/etcd-io/etcd
2. etcd Documentation. *Concurrency API Reference*. https://etcd.io/docs/v3.2/dev-guide/api_concurrency_reference_v3/
3. etcd Documentation. *Why etcd?* https://etcd.io/docs/v3.5/learning/why/